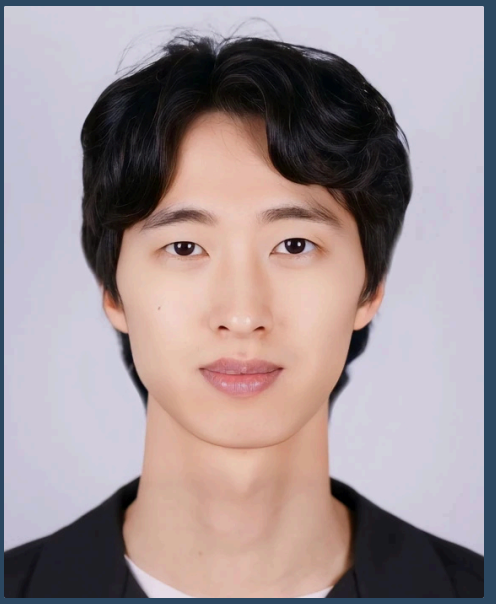
로봇 제어 · 비전 · 임베디드 신입

정승호 SEUNGHO JUNG

설계부터 구현, 디버깅까지
어느 단계도 가볍게 넘기지 않습니다

3095069@naver.com



프로젝트와 실습에서 사용한 기술

01 제어·임베디드

C · C++

STM32 · FreeRTOS · PLC

02 로봇·비전

Python · ROS2

OpenCV · YOLO · TFLite

03 설계·개발 도구

SolidWorks · Ansys · MATLAB

MuJoCo · Isaac Sim

학력과 교육, 자격 사항

기계공학 전공 지식을 바탕으로 로봇 제작과 제어를 학습했습니다

학력·교육

2026.04 – 2026.11 수료

대한상공회의소 서울기술교육센터

AI융합 로봇 SW개발자 3기

Python·Linux AI 프로그래밍
ROS/ROS2 통신·제어
Cortex-M·PLC·산업용 네트워크

2026.02 졸업

순천향대학교 기계공학과

학점 3.65 / 4.5

2019.02 졸업

대성고등학교

자격·어학

TOEIC 705점

2025.12

운전면허 1종 보통

2019.08 · 경찰청

초경량비행장치조종자

2019.09 · 한국교통안전공단

지그 위치에 동작을 맞추고 부품 안착까지 검사하는 로봇팔

Raspberry Pi5에서 AI·OpenCV·CAD로 지그 위치와 부품 상태를 확인합니다

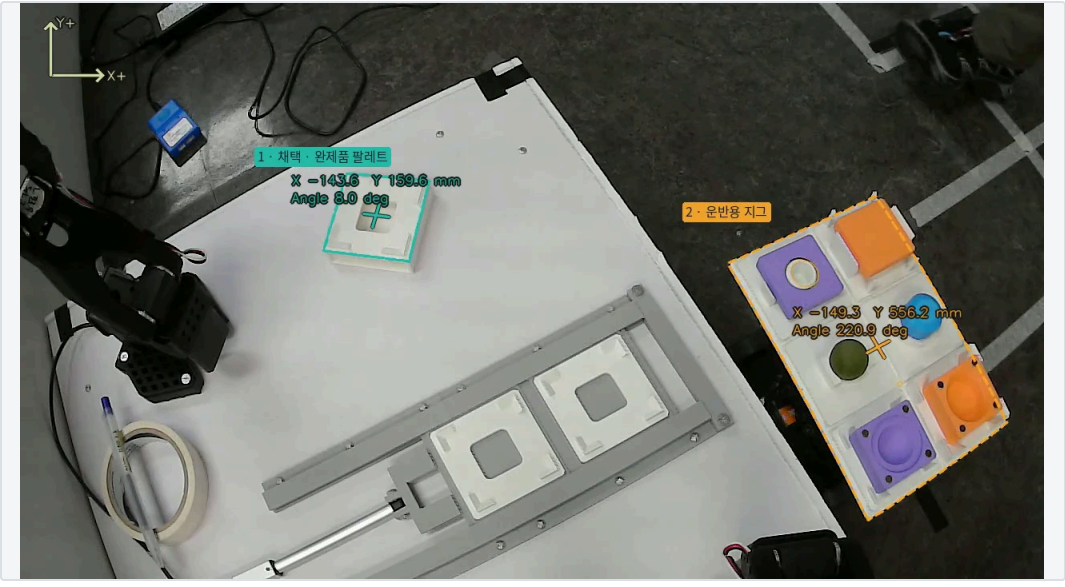


그림 1. 지그 이동 후 위치·방향을 다시 채택하는 화면

운반 로봇의 정차 위치가 달라지고 부품이 서로 가려도
저장한 동작으로 조립·하차 공정을 이어갑니다
카메라로 지그를 다시 측정해 동작 목표를 보정하고,
부품 종류와 안착 상태를 검사합니다

담당 역할

로봇팔 2대의 비전 보정·티칭 GUI·Raspberry Pi5 연동
학습 데이터·모델 비교와 실행 기준 검증

Python · OpenCV | YOLO · CAD 정합 | FK · IK | ROS2 Action | MuJoCo · Isaac Sim

기간	공정	장비	진행 상태
2026.09 – 2026.11	지그 도착 → 조립 → 완제품 하차	SO-101 2대 · Raspberry Pi5	프로젝트 진행 중

작업 요청부터 관절 목표까지 연결

ROS2 요청을 받아 지그를 측정하고, 지그의 이동·회전만큼 집게 목표와 관절값을 다시 계산합니다

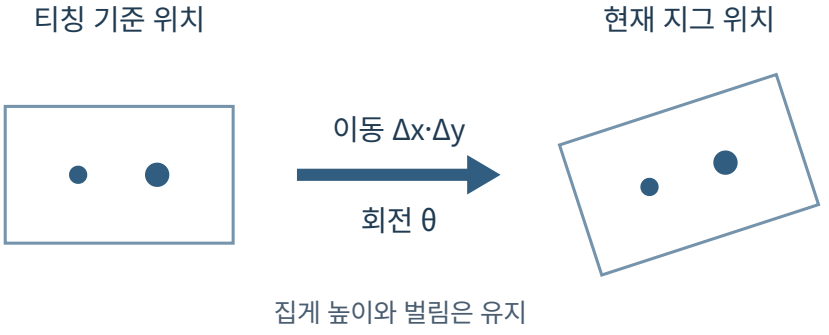


그림 2. 지그가 움직인 만큼 집게 작업점을 옮기고, 높이와 벌림은 유지

ROS2 요청과 결과 반환

조립·하차 요청을 받아 지그 측정·시작 검사 후 실행
진행·완료·실패 상태와 검사·오류 기록 반환

순기구학과 목표 변환

티칭 관절값과 TCP로 집게 자세 계산
지그 이동·회전만큼 집게 목표 보정

역기구학과 도착 확인

관절 범위·자세 연속성 확인 후 관절값 계산
모터 실측 각도로 목표 도착 확인

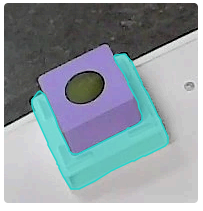
지그·팔레트 위치를 AI와 형상 검증으로 확정

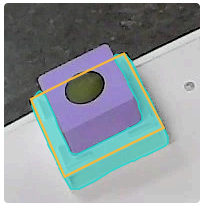
AI가 찾은 영역을 OpenCV·CAD 형상에 다시 맞춰, 위치·방향이 맞을 때만 채택합니다

AI 후보 영역

외곽·격자 검증

360° 방향 확인


보이는 AI 마스크


CAD 외곽 정합

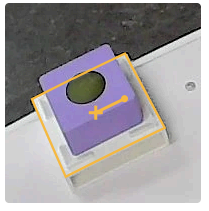

중심·방향 계산

그림 3. 운반용 지그(위)와 완제품 팔레트(아래)의 위치를 찾는 과정

운반용 지그 · 360° 방향 검증

AI 후보 영역을 외곽·격자·원형 특징으로 검증
첫 3관측이 **0.5mm·0.5°** 이내면 바로 채택
아니면 4초 판단 후 과반 그룹 중앙값, 5회 실패 시 중단

완제품 팔레트 · CAD 정합

AI가 찾은 영역에 설계 형상을 맞춰
가려진 부분까지 위치 계산
285장 중 284장에서 팔레트를 검출

팔레트 검출률	외곽선 검출	AI 인식 + 외곽선	AI 인식 + 설계 형상 맞춤 · 채택
촬영 사진 285장 · 최초 검출 시	**79.3%** (226/285장)	**80.7%** (230/285장)	**99.6%** (284/285장)

부품 종류와 제자리 안착을 따로 확인

티칭 GUI에서 기준 설정·스텝 편집과 측정·검사 결과를 함께 관리합니다

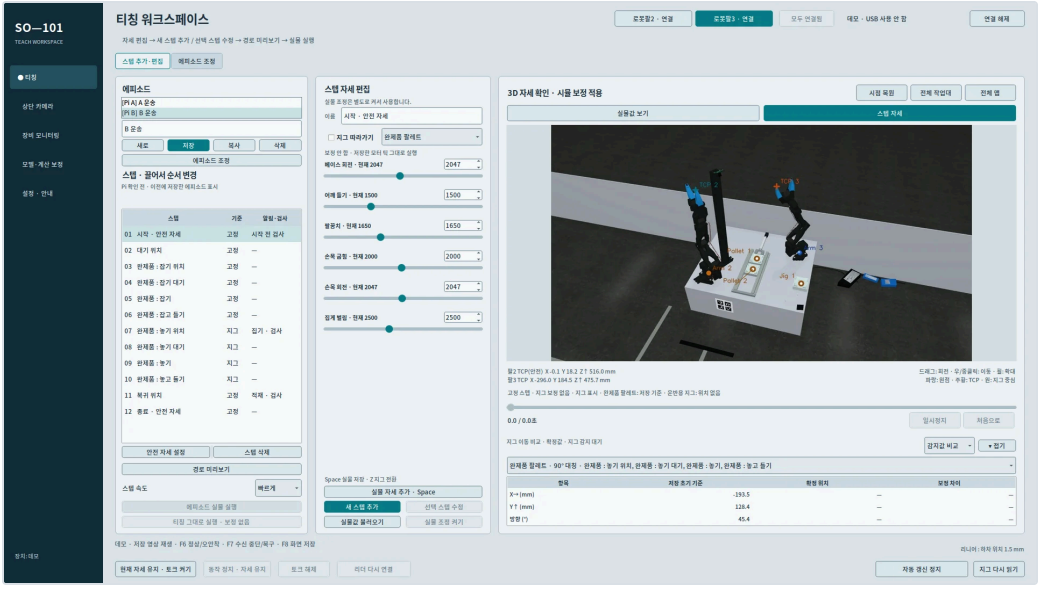


그림 4. 티칭 GUI · 스텝 편집·카메라 측정·검사

기준 설정
원점·TCP·카메라 좌표 기준

스텝 편집
고정 자세·지그 기준 보정 자세

측정·검사
지그 채택·부품 안착 결과

AI 라벨 · 무엇이 있는가
A/B 부품·조립 상태를 구분해 검사 위치에 연결

CAD 검사 · 제자리에 있는가
설계 형상·위치·방향으로 안착 상태 확인

같은 검사 위치에서 비교한 결과

방식	부품 구분	평균 처리
OpenCV	247/249	0.937초
AI 라벨 + CAD 채택	**249/249**	**1.592초**

부품 구분 오류 2건을 없애 **249개 모두 정확히 구분**

실물·가상 데이터로 학습하고 Pi에 맞게 경량화

지그·단품·부분 조립체·완제품 13개 클래스를 구분하고, Raspberry Pi5에서 실행 형식과 모델 크기를 정했습니다

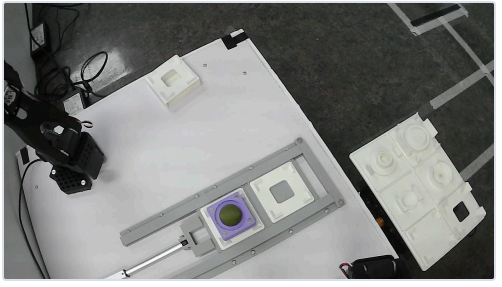


그림 5. 실물 사진 · 부품 종류와 보이는 윤곽

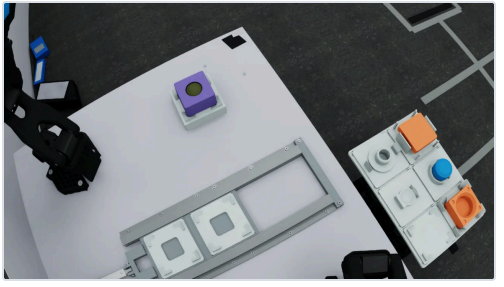


그림 6. Isaac Sim 가상 데이터 · 배치·적재·조명 변화

13개 클래스

운반용 지그 · 팔레트 · 리니어 지그 · A/B 단품 3종 · A/B 부분 조립체 · A/B 완제품

가상 데이터

오안착 137장·경계 사례 45장 포함, 배치·적재·조명 변화

학습·검증·테스트 분리

Raspberry Pi5 실행 기준

순차 학습 채택

최저 클래스 IoU **96.72%**

평균 IoU가 비슷한 혼합·순차 중 편차 0.60으로 가장 안정적

FP16 채택

처리 시간 **61%** 감소

1.341초 → 0.523초 · 원본 대비 IoU 차이 0.02점

YOLO26n 채택

처리 시간 **53%** 감소

s 대비 IoU 차이 0.15점 · 메모리 38% 절감

실물 31장·169개 · 신뢰도 50% · 처리 시간은 Pi5 중앙값

오안착 원인을 실행 기록과 판정 영상으로 확인

실행 기록과 같은 실행의 판정 영상을 연결해 정지·에러 사유를 확인합니다

01 · 실행 기록 요약

시각	기록
19:50:15	A 조립 작업 접수
19:50:23	지그 위치·방향 확정
19:50:25	**중심 이탈 7.8mm · 안착 불합격**
19:50:27	판정 영상·오류 기록 저장
19:50:27	에피소드 중단

중심 이탈 7.8mm > 허용 5mm
제자리를 벗어난 부품을 검출해 작업 중단

같은 실행

02 · 저장된 판정 영상

그림 7. 붉은 영역 · A 하단 오안착, 중심 이탈 7.8mm

스마트폰으로 조작하는 이동형 로봇

4륜 이동체와 로봇팔을 결합해 스마트폰으로 주행과 물품 회수 동작을 제어합니다

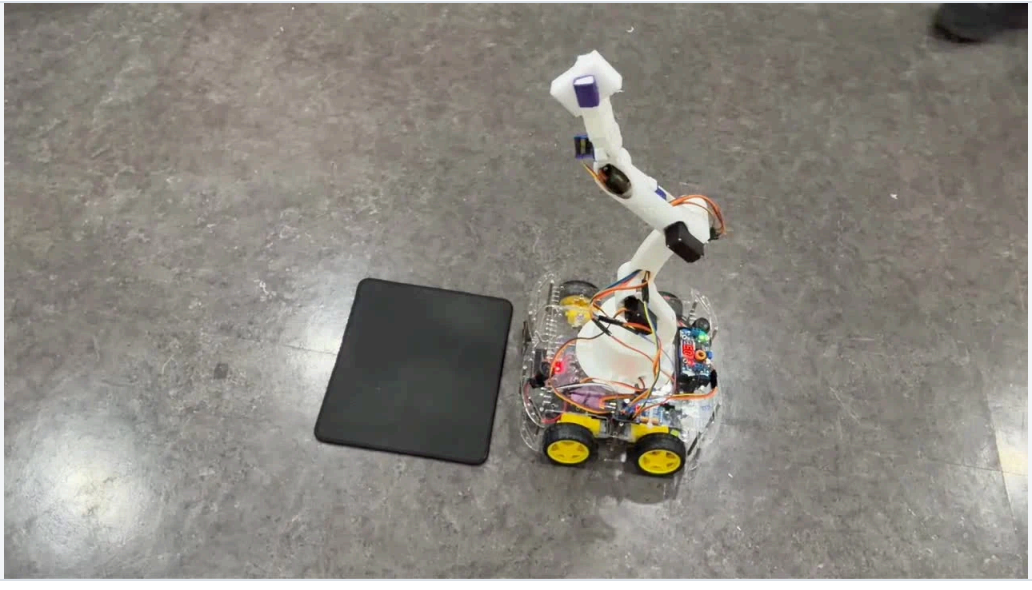


그림 8. 원격 물품 회수·운반 로봇 시연품

이동체와 로봇팔을 결합한 물품 회수·운반 시연품입니다
앱으로 주행과 관절을 수동 조작하고, 저장한 동작을 재생합니다

> **담당 역할**
> 프로젝트 리더 · 로봇팔 제작·제어, STM32 펌웨어 구축·통합
> 통신 패킷 규격 설계와 앱 연동

STM32F411 | C · FreeRTOS | Flutter | Bluetooth

수행 시기	제어 보드	구동 구성	구현 범위
2026.07 · 1주	**STM32F411**	**4WD · 서보 6개**	**수동 조작 · 티칭**

로봇팔 제어와 시스템 통신 구조

16바이트 통신 규격으로 서보 제어·보정·티칭 데이터를 주고받고 저장합니다

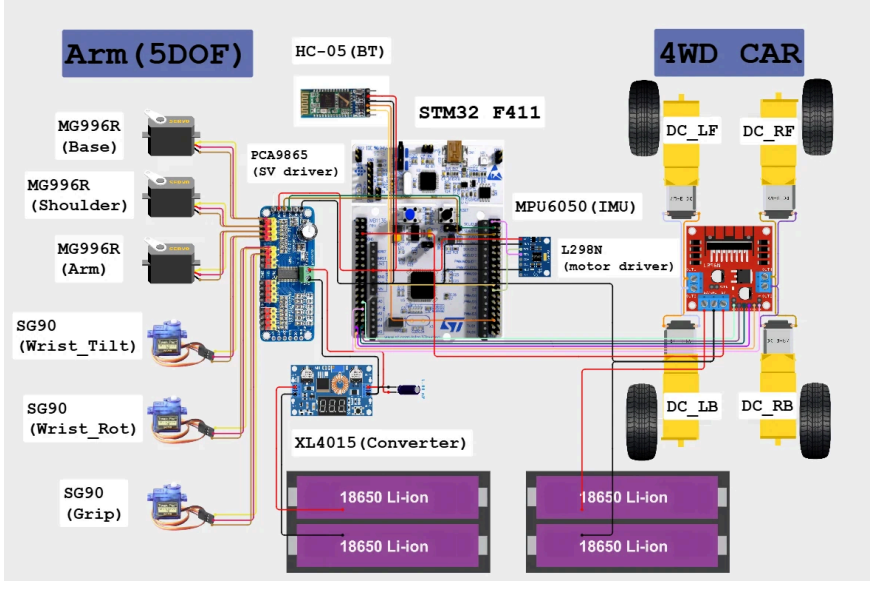


그림 9. 로봇팔·이동체의 전체 배선도

16바이트 명령·응답

주행·로봇팔·티칭·설정을 모드로 구분
앱 요청과 펌웨어 응답을 연결해 처리 상태 확인

서보 제어와 동작 저장

PCA9685로 서보 6개 제어
Flash에 티칭·설정 저장, 최대 **12개 시퀀스 × 30개 웨이포인트**

동작 간 간섭 방지

로봇팔 이동·Flash 작업 중 주행 잠금
500ms 동안 유효한 주행 명령이 없으면 이동체 정지

조작 기능과 구동 문제의 개선

부하 시 떨림과 급격한 움직임을 줄이고, 관절별 원점·가동 범위를 보정했습니다

01 주행 02 로봇팔 03 원점 보정 04 티칭

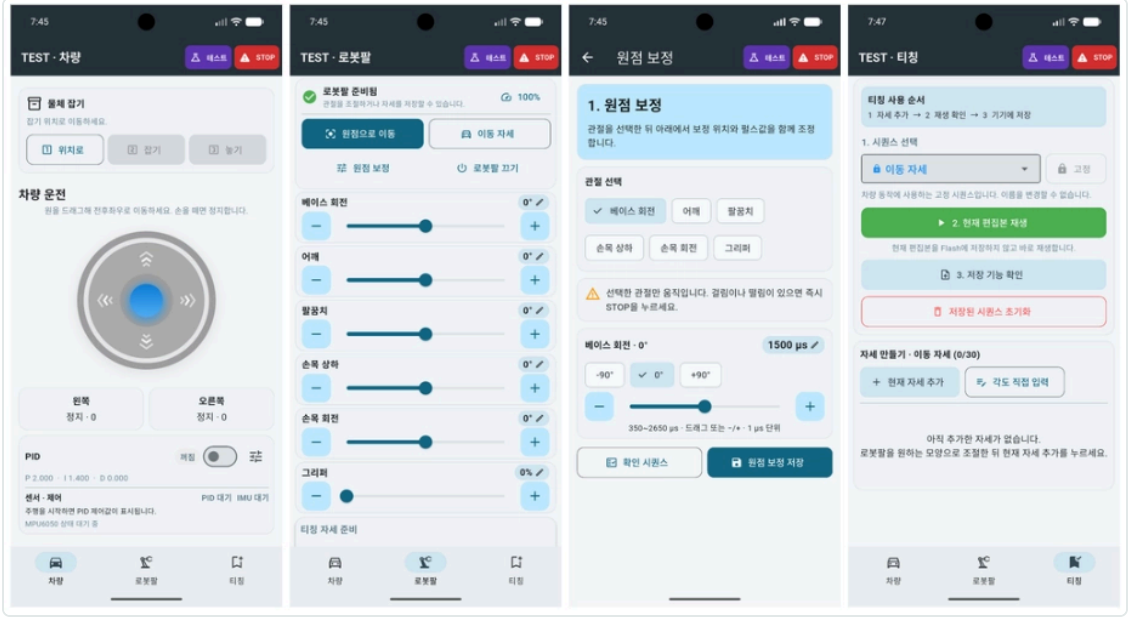


그림 10. 앱 TEST 모드 화면 · 주행 · 로봇팔 · 원점 보정 · 티칭

문제	개선 내용
부하가 걸릴 때의 떨림	전원부 커패시터 추가, 토크 여유가 있는 서보로 교체 **부하 시 로봇팔 떨림 감소**
관절의 급격한 움직임	**20ms 주기 S-curve**로 관절 구동
관절 원점·가동 범위 보정	**−90°·0°·+90° 3점 보정**으로 원점·리미트 설정 서보별 실측 펄스값을 저장해 관절 제어에 반영

객체 탐지와 지면 분할을 이용한 보행 보조

Raspberry Pi5에서 영상 처리, 객체 탐지, 지면 분할을 함께 실행했습니다

그림 11. Raspberry Pi5에서 실행한 객체·지면 인식 화면

영상에서 사람·차량·장애물을 탐지하고 인도와 차도를 구분합니다
보행 경로에 가까운 대상을 판단해 음성으로 알려 줍니다

> **담당 역할**
> 학습 데이터 라벨 통합, 모델 학습·변환, 실행 환경 구성
> 보행·차량 ROI와 알림 판단 로직 구현, 녹화 영상으로 동작 확인

Python · OpenCV | YOLO26n | TFLite INT8

수행 시기	실행 장치	객체 라벨 통합	지면 라벨 통합
2026.05 · 11일	**Raspberry Pi5**	**29종 → 10종**	**23종 → 7종**

Raspberry Pi5 실행 환경에 맞춘 조정

입력 해상도와 모델 형식을 조정하고, 보행 경로와 차량 접근을 판단할 영역을 정했습니다

입력 해상도 축소와 모델 변환

입력 해상도 **1920×1080 → 512×288**, 모델은 **INT8 TFLite**로 변환

보행·차량 ROI 분리

보행 ROI는 발 앞쪽인 화면 하단 중앙에 배치
옆에서 접근하는 차량은 더 넓은 **차량 ROI**로 판단

녹화 영상 기반 동작 시험

녹화 영상을 재생하며 추론 동작 확인
추론 빈도는 **객체 4fps, 지면 3fps**로 제한

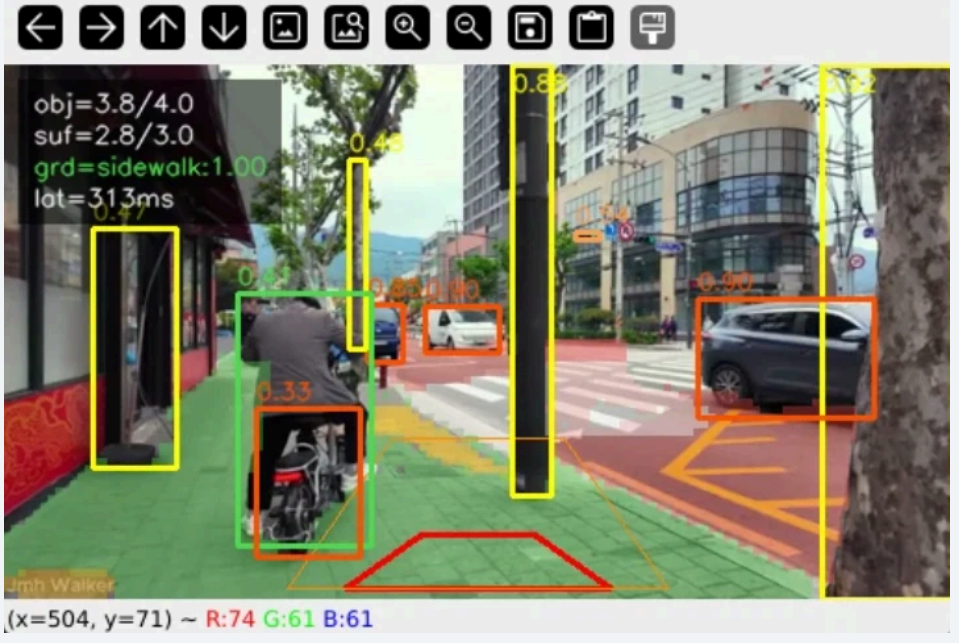


그림 12. 빨간색: 보행 경로 · 주황색: 차량 접근 영역

소재 공급부터 창고 적재까지의 공정 제어

MPS 트레이너에서 소재 분류와 1–4층 창고 적재·비움 동작을 구현했습니다

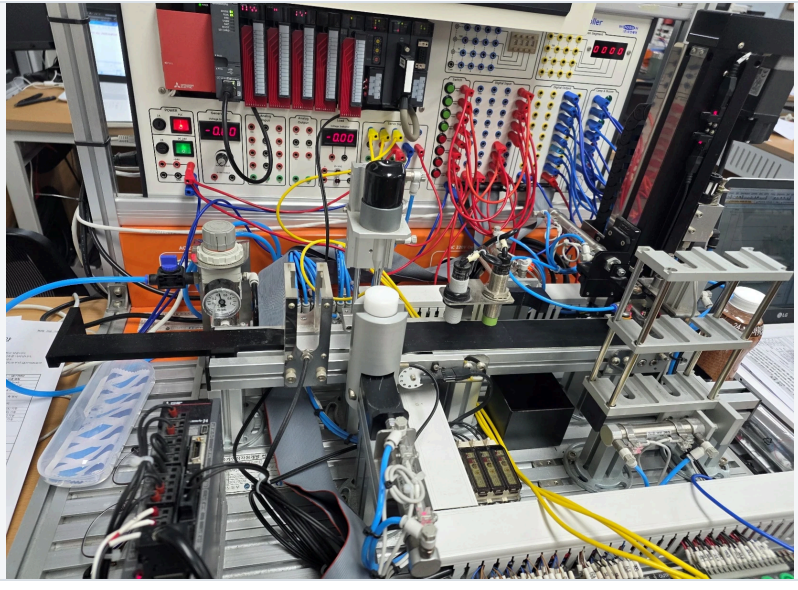


그림 13. MPS 트레이너 설비

컨베이어·실린더·서보모터로 공급·가공·분류·저장 공정을 연결했습니다 여러 장치를 병렬로 운전하고, 수동 모드에서는 개별 제어할 수 있게 했습니다

담당 역할

병렬 처리 로직과 HMI 화면 작성

수동 제어 코드 통합, 메모리 상태 관리로 오동작 해결

Mitsubishi Q PLC | GX Works2 | GT Designer3

수행 시기	PLC	래더 프로그램	HMI 작화
2026.06 · 3일	Mitsubishi Q 시리즈	GX Works2	GT Designer3

병렬 시퀀스의 상태 관리와 코드 통합

저장 가능 여부를 판단하는 메모리와 실제 적재 정보를 분리해 시퀀스 간 간섭을 줄였습니다

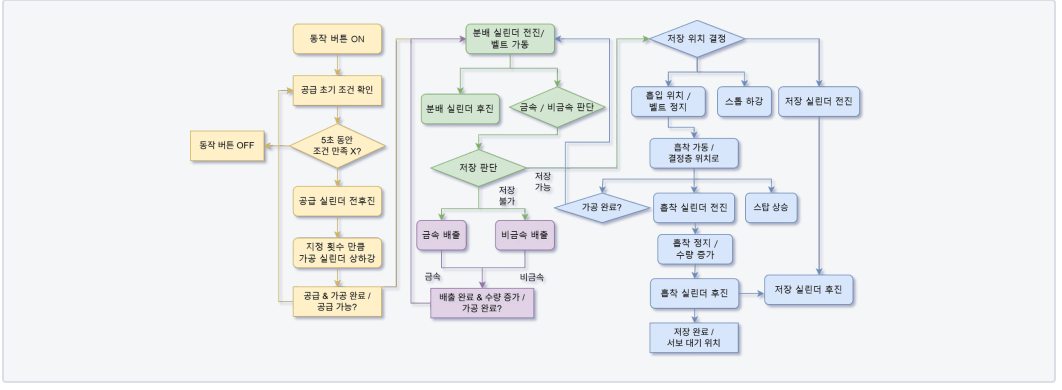


그림 14. 저장 판단과 적재 동작의 병렬 시퀀스

그림 15. 자동 운전·적재 상태

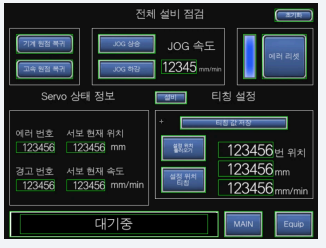


그림 16. 위치·속도 티칭

문제	메모리 저장·초기화 시점이 맞지 않아 **이전 동작의 상태가 다음 시퀀스의 판단에 영향**
개선	**판단용·적재용 메모리 분리**, 동작 조건과 완료 시점에 맞춰 저장·초기화
통합 시험	조건별 동작 확인, **통합 코드에서 겹치는 메모리 주소 정리**
배운 점	코드 통합 전 **메모리 사용 주소 합의**가 중요함

피에조 센서를 이용한 노크 위치 탐지

센서 간 시간 차와 시간·주파수 영역 특징을 입력으로 사용해 기계학습 모델을 비교했습니다

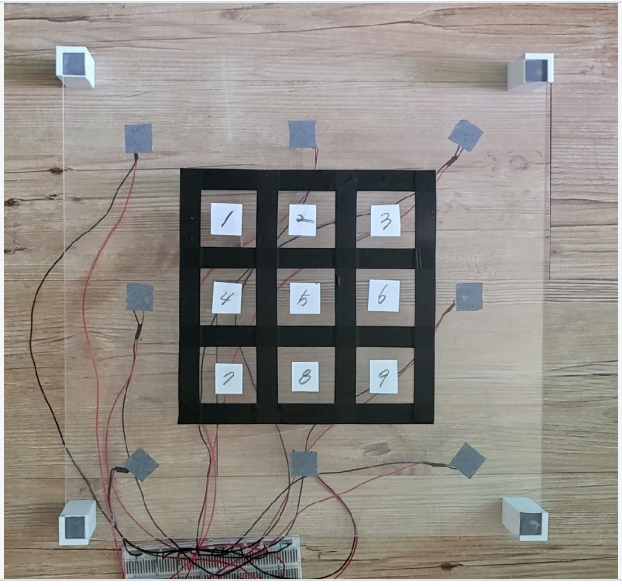


그림 17. 피에조 센서 8개를 붙인 아크릴판 실험 장치

아크릴판을 두드릴 때 발생하는 진동을 8개의 피에조 센서로 수집합니다
수집한 신호를 분석해 노크 위치를 9개 구역으로 구분합니다

담당 역할

노크 위치 탐지 주제 제안
MATLAB 전처리·특징 추출, 모델 비교, 실시간 탐지 프로그램 구현

MATLAB | FFT · 신호 처리 | K-NN · SVM · ANN

논문 제출	센서	수집 데이터	기록 조건
2023.05	**피에조 센서 8개 · DAQ**	**9구역 × 140회**	**2,500Hz · 0.4초**

입력 특징별 K-NN 검증 결과

같은 아크릴판에서 수집한 데이터를 학습 70%, 검증 30%로 무작위 분할해 비교했습니다

신호 수집부터 실시간 탐지까지

61Hz 하이패스 필터와 데이터 평탄화 후 특징을 추출·정규화해 모델 학습

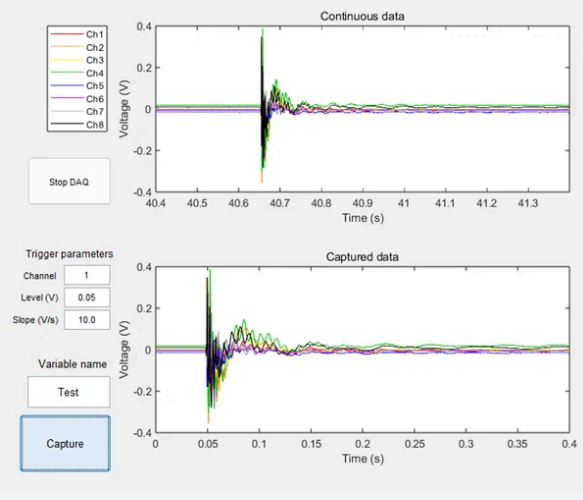


그림 18. 연속 신호와 트리거 기록 화면

K-NN 검증 정확도

입력 특징	정확도
센서 간 피크 시간 차	90.741%
시간 영역 특징	96.296%
주파수 영역 특징	97.884%

이동 경로와 사진을 연결한 여행 기록

여행 사진을 이동 경로와 함께 돌아보고 싶어 만든 웹앱입니다

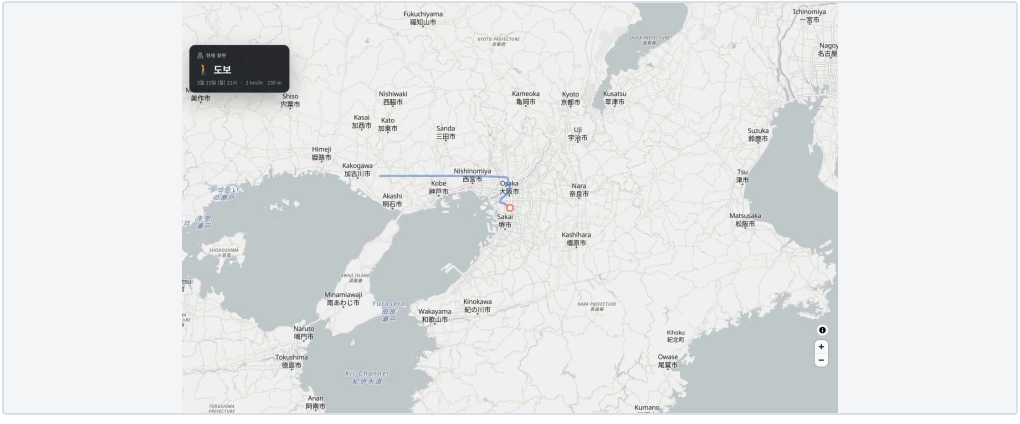


그림 19. 경로 보기 · 이동 기록을 따라가는 지도

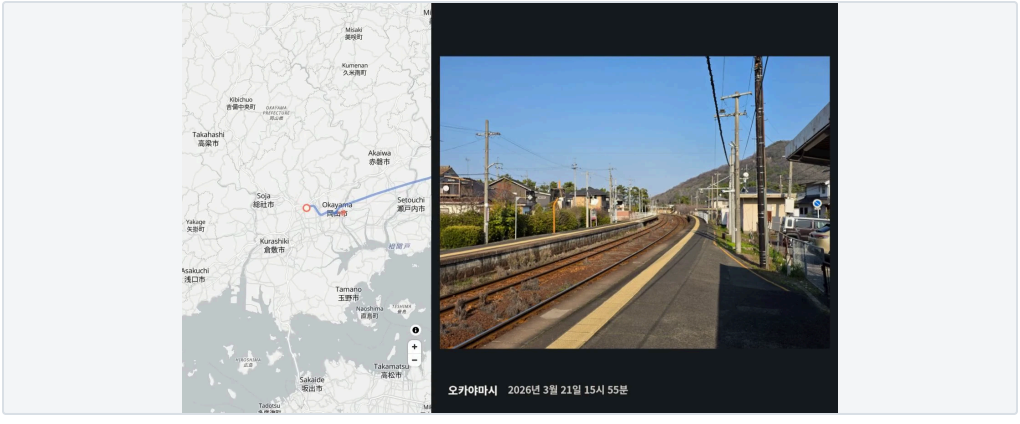


그림 20. 사진 여정 · 촬영 순서로 이어지는 장면

구현 기능

Google Timeline 이동 기록에 사진·영상 연결

경로 보기·사진 여정 두 모드, PC·모바일 화면별 배치

React · TypeScript | MapLibre | Node.js

담당 역할

화면·기능 기획과 동작 기준 정의

실제 여행 기록으로 오류 재현, 수정 전후 동작 비교

사용 중 발견한 문제와 개선 과정

재생 모드 전환, 위치 정보 판단, 로컬 파일 처리 과정을 개선했습니다

01

이동 기록·사진

Timeline JSON과
사진·영상 정보 입력

›

02

경로·장면 구성

촬영 시간·위치 연결
여행 선택과 경로 계획

›

03

경로·사진 재생

모드별 재생 위치 유지
화면 크기에 맞춘 배치

모드별 재생 위치 유지

모드를 바꿔도 재생 위치를 유지하는 기준 정의
실제 여행 기록으로 재생·전환 동작 검증

불확실한 이동 기록 처리

이동 기록이 끊기거나 GPS와 위치 정보가 크게
다를 때 잘못된 이동 수단·장소 표시 방지

원본 파일 로컬 처리

사진·Timeline 원본은 외부 업로드 없이
로컬에서 처리